

Lecture 5

Getting Started with Django

Installing the framework, creating the first project, understanding the files, and running the development server.

Dr. Mohamed Nour Eldien



Goal: every student leaves the lab able to create and run a fresh Django project.

Why do we need a framework?

Students already know Python syntax; now they need a faster way to build full web apps.

Core classroom idea

- Writing every web feature from zero wastes time.
- A framework provides ready-made structure and common tools.
- It reduces repeated code such as routing, forms, and admin tasks.
- It helps teams follow one organized style.

Simple example

Imagine building a student portal. Without a framework, you would manually connect URLs, forms, database logic, and templates. With Django, many of these pieces are already organized for you.

What is Django?

A high-level Python web framework

Definition

- Django is built using Python.
- It is designed for rapid development.
- It follows a structured architecture called MVT.
- It includes admin panel, ORM, routing, templates, and security features.

Why teachers like it for second-year students

It is practical, readable, and close to real industry work. Students can build useful web apps early without getting lost in too much low-level setup.

- Django Basics :

- Virtualenv
- Django & Git , Github
- Models
- Views
- Templates
- Forms
- Admin panel
- Database

- Django Packages

- RestAPI [Authentication]
- Translation [en-ar]
- Admin Panel Customization [themes]
- Deploy :
 - Heroku
 - DigitalOcean
 - Ssl Certificate
 - Connect Domain

Why Django is a strong teaching choice

It is fast to learn, but also real enough for serious projects.

Technical strengths

- Clean project structure
- Built-in admin interface
- ORM instead of raw SQL at the start
- Good documentation and a huge community

Teaching strengths

It lets us connect Python, web basics, templates, URLs, models, and forms inside one coherent environment. That is perfect for a semester plan.

Django follows the MVT pattern

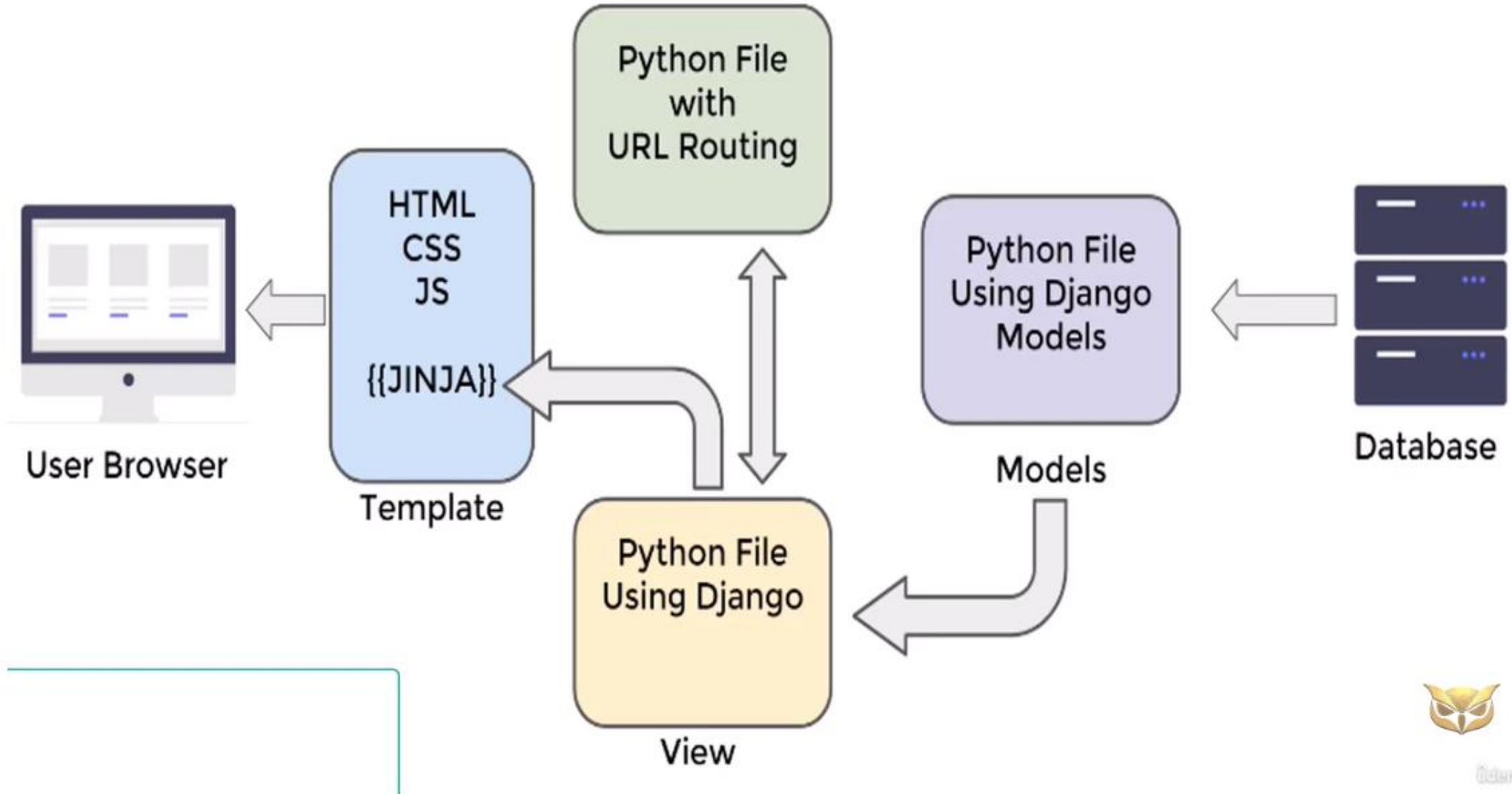
Model • View • Template

Three main pieces

- Model: handles data and database tables.
- View: receives requests and decides what logic to run.
- Template: controls what the user sees in HTML.

Important teaching note

MVT in Django is close to MVC in the general web-development discussion. For our course, focus on the role of each file rather than memorizing terms only.



A simple Django request flow

Show the path from browser to response in one straight line.

Browser

The user types a URL or submits a form.

URL + View

Django matches the path and runs a Python function.

Model

If needed, data is read from or written to the database.

Template

HTML is prepared for the user.

Response

The browser shows the result.

Part 1 • Tools and Installation

Python, pip, VS Code, terminal, virtual environment, and Django package.



In this part, slow down and connect every command to a real classroom demonstration.

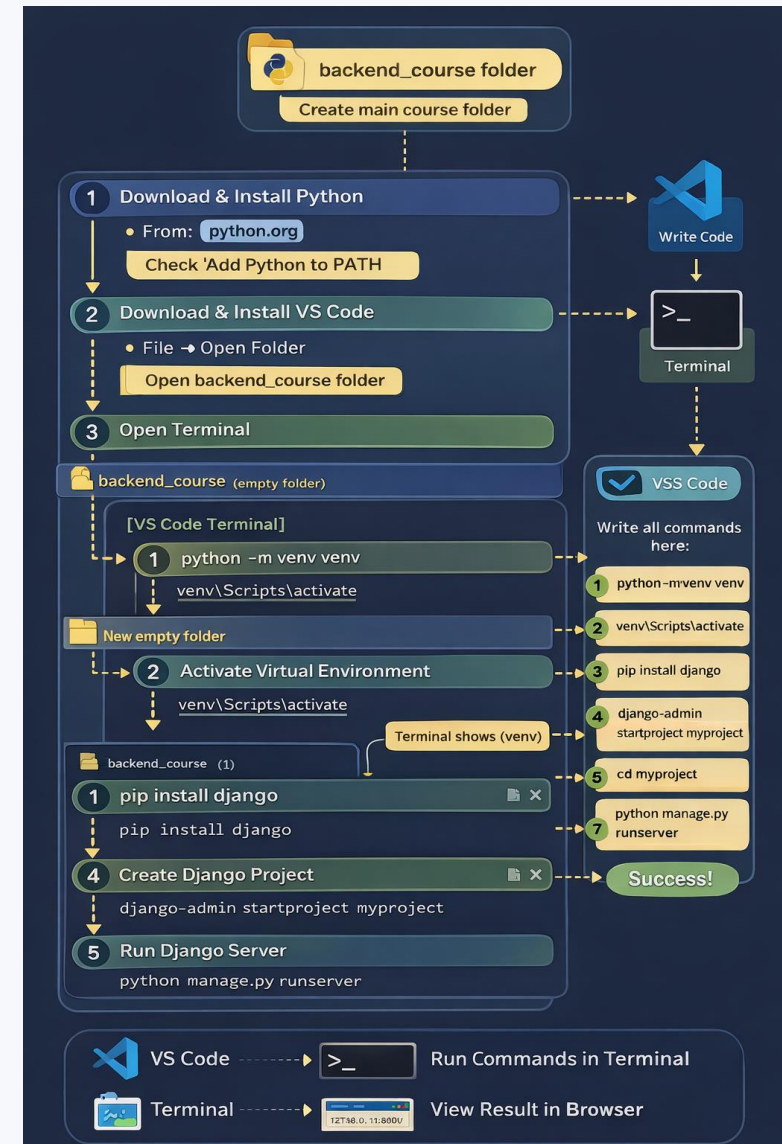
Python + Django Setup Workflow

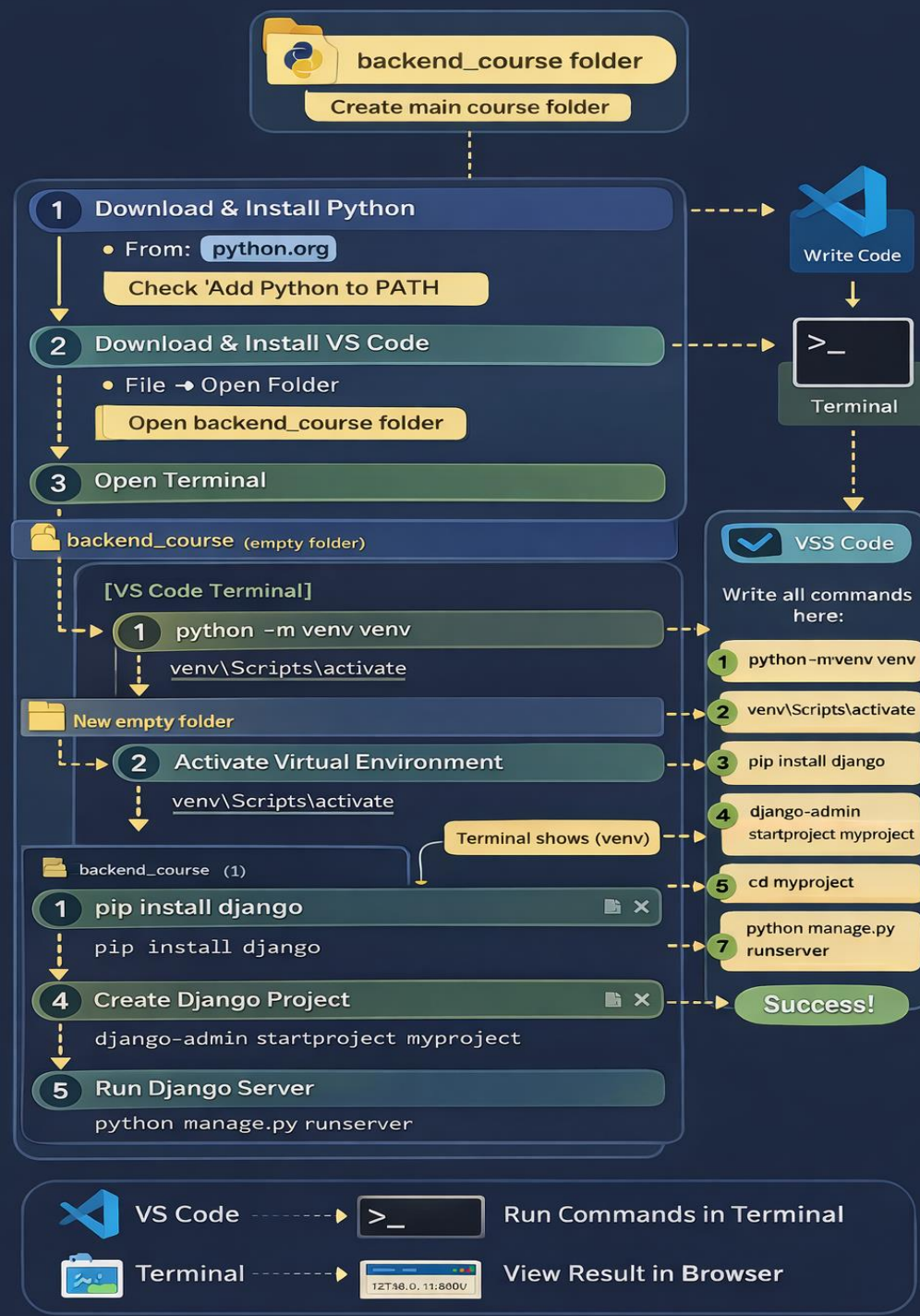
Open VS Code • Create a virtual environment • Install Django • Create the project • Run the local server

- What each step does
- Why the step is necessary
- Where to write each command
- How the project folder changes after key steps

Key idea

VS Code = write code
Terminal = run commands
Browser = view result





Workflow overview

From local setup to a running Django website

- 1. Open the backend_course folder in VS Code.
- 2. Open the integrated terminal.
- 3. Verify that Python is installed.
- 4. Create and activate a virtual environment.
- 5. Install Django with pip.
- 6. Create a Django project and enter its folder.
- 7. Run the local development server.
- 8. Open the project in the browser at <http://127.0.0.1:8000/>

Folder status before setup

backend_course/

```
1 python --version
```

```
1 python -m venv venv
```

```
1 venv\Scripts\activate
```

```
1 pip install django
```

```
1 django-admin startproject myproject
```

Step 1 — Open VS Code

Action

1 File → Open Folder → backend_course

Where to do it

Inside Visual Studio Code.

Why this step matters

- Opening the whole folder lets VS Code manage the project as one workspace.
- The Explorer panel will show files and folders clearly.
- The integrated terminal will usually open in the correct location.
- This becomes very important later when Django creates many files.

Folder at this moment

backend_course/

Remember

This step prepares the next one. The workflow should be followed in order.

Step 2 — Open Terminal

Action

```
1 Terminal → New Terminal
```

Where to do it

Inside Visual Studio Code, at the bottom panel.

Why this step matters

- The terminal is where you run Python and Django commands.
- You will use it to create the virtual environment, install packages, and run the server.
- Keeping the terminal inside VS Code is easier for students because code, commands, and output stay in one place.

Folder still unchanged

```
backend_course/
```

Remember

This step prepares the next one. The workflow should be followed in order.

Step 3 — Check Python

Action

```
1 python --version
```

Where to do it

Write the command in the VS Code terminal.

Why this step matters

- This confirms that Python is installed correctly.
- Django cannot work until Python is available on the computer.
- It also confirms that the terminal recognizes the python command.
- If the command is not recognized, Python may not be installed or PATH may not be configured correctly.

Folder change

No new files yet.

Remember

If Python is not recognized, fix the installation before moving on.

Step 4 — Create the virtual environment

Action

```
1 python -m venv venv
```

Where to do it

Write the command in the VS Code terminal while you are inside `backend_course`.

Why this step matters

- A virtual environment creates an isolated Python environment for this project only.
- It keeps the project's packages separate from other projects on the same computer.
- This prevents version conflicts when different projects need different package versions.
- It makes the project easier to move to another machine later.

Folder after Step 4

```
backend_course/  
├── venv/
```

Remember

Creating the environment does not activate it. Activation is the next step.

Step 5 — Activate the virtual environment

Action

```
1 venv\Scripts\activate
```

Where to do it

Write the command in the VS Code terminal on Windows.

Why this step matters

- Activation tells the terminal to use the project's private Python environment now.
- After activation, installed packages go into this environment instead of the global system.
- You should see (venv) at the start of the terminal line.
- This is the sign that the environment is active and ready for Django installation.

Folder after Step 5

```
backend_course/  
├── venv/  
Terminal prompt shows: (venv)
```

Remember

Look for (venv) in the terminal prompt as proof of activation.

Step 6 — Install Django

Action

```
1 pip install django
```

Where to do it

Write the command in the VS Code terminal after activation.

Why this step matters

- pip is Python's package manager; it downloads and installs external libraries.
- Django is a Python web framework, not part of the basic Python installation.
- We install Django because it provides routing, views, templates, models, forms, admin tools, and database support.
- Installing Django inside the virtual environment keeps the setup clean and project-specific.

Folder after Step 6

```
backend_course/  
├── venv/  
│   └── Django package files installed here
```

Remember

Install Django only after the environment is active.

Step 7 — Create the Django project

Action

```
1 django-admin startproject myproject
```

Where to do it

Write the command in the VS Code terminal while still in `backend_course`.

Why this step matters

- This command creates the standard Django project structure automatically.
- It generates the main configuration files needed to start development.
- You do not need to build the project structure manually from scratch.
- This saves time and reduces mistakes for beginners.

Folder after Step 7

```
backend_course/  
├── venv/  
└── myproject/  
    ├── manage.py  
    └── myproject/  
        ├── settings.py  
        ├── urls.py  
        ├── asgi.py  
        └── wsgi.py
```

Remember

This step prepares the next one. The workflow should be followed in order.

Initial project tree

Use this slide to point at each file with the cursor.

myproject/

Outer project
folder
Contains
manage.py and
the inner package

manage.py

Command
entry point
Used for
runserver,
migrate,
startapp

myproject/

Inner project
package
Contains settings
and URLs

Inside the inner folder

settings.py
urls.py
asgi.py
wsgi.py

Step 8 — Enter the project folder

Action

```
1 cd myproject
```

Where to do it

Write the command in the VS Code terminal.

Why this step matters

- The file `manage.py` is inside the `myproject` folder.
- Most Django commands must be run from the folder that contains `manage.py`.
- This moves the terminal into the correct location for the next command.

Terminal location after Step 8

```
backend_course\myproject>
```

Remember

If `manage.py` is not found, you are probably in the wrong folder.

Step 9 — Run the Django server

Action

```
1 python manage.py runserver
```

Where to do it

Write the command in the terminal inside the myproject folder.

Why this step matters

- This starts Django's local development server.
- The server allows your browser to connect to the project running on your own computer.
- It lets you test pages, URLs, templates, and backend logic during development.
- This is a development server used for learning and local testing.

What happens now?

No major folder change.
The project becomes available locally in the browser.

Remember

Keep the server running while you test the project in the browser.

Step 10 — Open the browser

Action

```
1 http://127.0.0.1:8000/
```

Where to do it

Write the URL in Chrome, Edge, Firefox, or any browser.

Why this step matters

- 127.0.0.1 means your own computer (localhost).
- Port 8000 is the default port used by Django's development server.
- If the setup is correct, you will see the default Django welcome page.
- This confirms that Python, the virtual environment, Django, the project structure, and the server are all working together.

Final status

```
backend_course/  
├── venv/  
└── myproject/  
    ├── manage.py  
    └── myproject/  
Browser opens: http://127.0.0.1:8000/
```

Remember

If the page opens successfully, the setup chain worked end to end.

One-sentence summary for students

Write code in VS Code → run commands in Terminal → view the result in the Browser

- Python is the programming language.
- The virtual environment protects the project and keeps packages isolated.
- Django is the framework that provides the web application structure.
- `manage.py` is the command entry point for the Django project.
- The browser shows the running local website at `127.0.0.1:8000`.

Final project view

```
backend_course/  
├── venv/  
└── myproject/  
    ├── manage.py  
    └── myproject/  
        ├── settings.py  
        └── urls.py
```


Tool checklist before writing code

Make students verify the environment before typing framework commands.

Required items

- Python installed
- pip available
- VS Code installed
- Terminal or command prompt working
- Internet connection for package installation

Teacher note

Spend real time here. In many labs, this is where students get blocked. A working environment saves huge time later.

Check the Python version

First command to verify the interpreter



```
1 python --version
2 # sometimes
3 python3 --version
```

What to explain

- Run the command in terminal.
- If Python is installed, a version number appears.
- If command not found appears, installation or PATH is the problem.

Show it live in class before moving on.

Installing Python – Step 1

Download from the official website

What students should do

- Open python.org in the browser.
- Download Python 3 for your operating system.
- Prefer the stable release shown on the home page.
- Start the installer normally.

Why the official site matters

The official site gives the correct installer, documentation, and trusted version. This reduces confusion and avoids broken package setups.

Installing Python – Step 2

The important checkbox on Windows

Critical point

- On Windows, check “Add Python to PATH”.
- Then continue the installation.
- Without PATH, terminal may not recognize python.

How to explain PATH simply

PATH is like a list of places the operating system searches when you type a command. If Python is not in that list, the terminal cannot find it easily.

Verify pip

pip is Python's package installer



```
1 pip --version
2 # sometimes
3 pip3 --version
```

What to explain

- pip installs packages such as Django.
- It should show a version and Python path.
- If pip is missing, Python installation may be incomplete.

Install VS Code

A lightweight editor that works well with Python and Django

What students do

- Download from code.visualstudio.com.
- Run the installer and keep default options.
- Open VS Code after installation.
- Allow it to add desktop or context-menu shortcuts if needed.

Why VS Code is enough for this course

It gives us file navigation, syntax coloring, terminal, and extensions in one place. Students can see code and run commands without switching tools constantly.

Install the Python extension in VS Code

This improves running, linting, and editing Python files.

Steps

- Open the Extensions sidebar.
- Search for Python.
- Install the Microsoft Python extension.
- Restart VS Code if needed.

Benefit in class

Students get syntax help, interpreter selection, and a smoother workflow. It reduces confusion when opening .py files and virtual environments.

Create a course workspace folder

Keep each project inside a clean parent folder.

Recommended classroom structure

- Create one main folder such as backend_course.
- Inside it, create another folder for today's project.
- Avoid saving projects in random places on the desktop.

Why organization matters

Later we will have many apps, templates, and environments. A clean folder structure makes debugging and revision much easier.

Open the folder in VS Code

File → Open Folder is better than opening individual files.

Explain this workflow

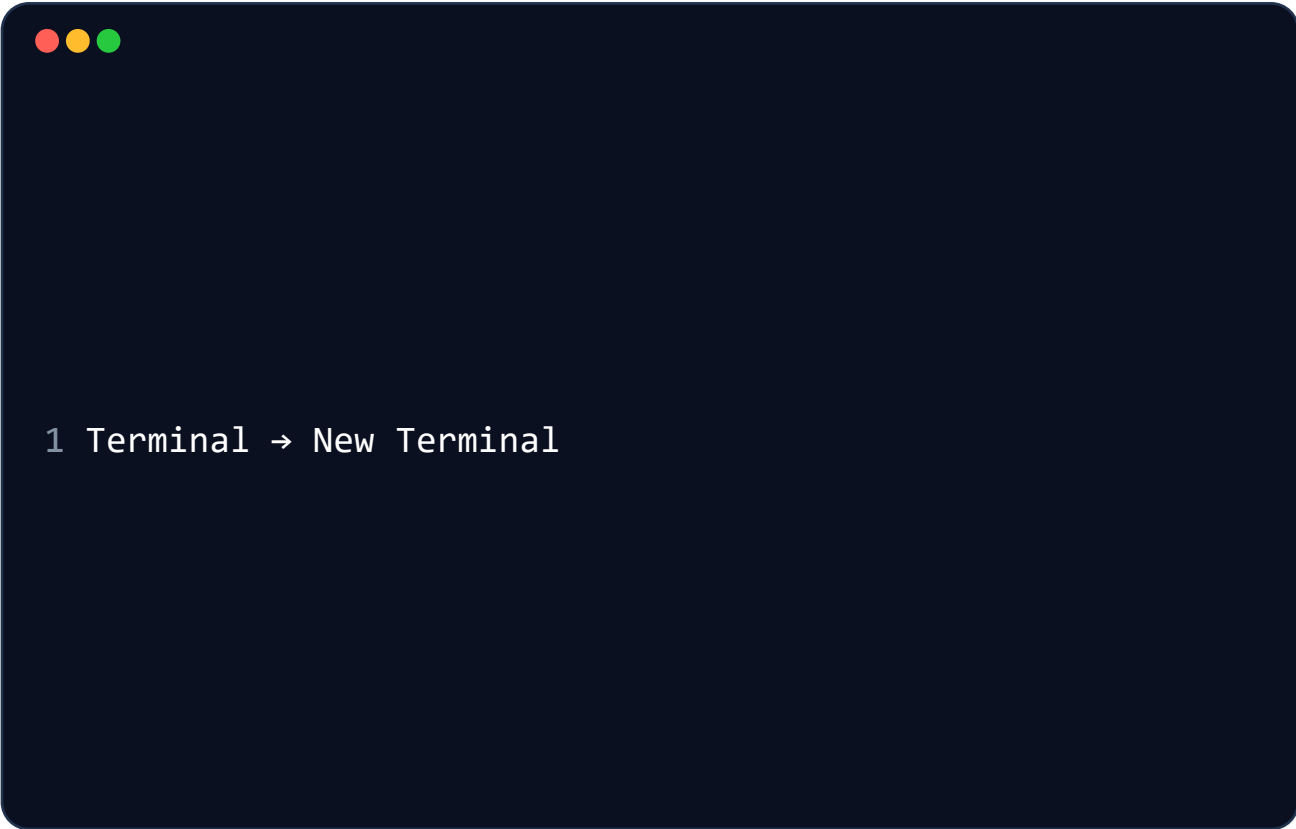
- Open the whole project folder, not one file only.
- The Explorer will show the full project tree.
- The integrated terminal will open inside that folder.

Why it matters for Django

Django commands such as `startproject` and `runserver` depend on where the terminal is opened. The working directory matters.

Open the integrated terminal

Use the terminal inside VS Code instead of switching to many windows.



```
1 Terminal → New Terminal
```

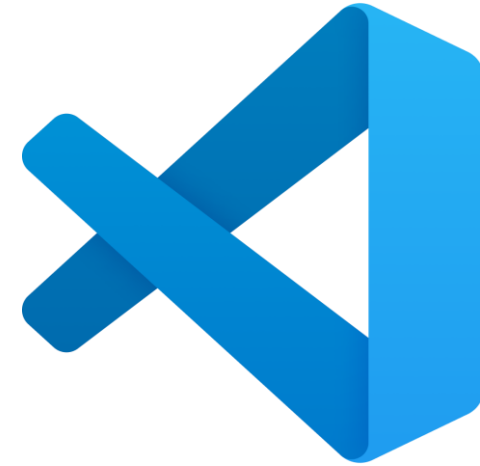
What to explain

- The prompt opens inside the current folder.
- Most Django commands are typed here.
- Students should learn to read the current path before running commands.

Part 2 • Virtual Environment

Students must understand isolation before package installation.

In this part, slow down and connect every command to a real classroom demonstration.



What is a virtual environment?

A separate Python environment for one project

Core meaning

- It isolates project packages from the system-wide Python.
- Different projects can use different package versions.
- It keeps the machine cleaner and more predictable.

Simple analogy

Think of it as a private lab table for one project. The tools on that table belong to this project only.

Create a virtual environment

Use the built-in venv module

A dark-themed terminal window with three colored window control buttons (red, yellow, green) in the top-left corner. The command 'python -m venv venv' is entered on the first line, preceded by a line number '1'.

```
1 python -m venv venv
```

What to explain

- python runs the interpreter.
- -m means run a module as a script.
- venv is the module name.
- The last venv is the folder name that will be created.

Activate the environment on Windows

The prompt usually changes after activation



```
1 .\venv\Scripts\activate
```

What to explain

- This is the common Windows activation command.
- After activation, the prompt often starts with (venv).
- That visual cue is very important.

Activate the environment on macOS / Linux

A slightly different command path



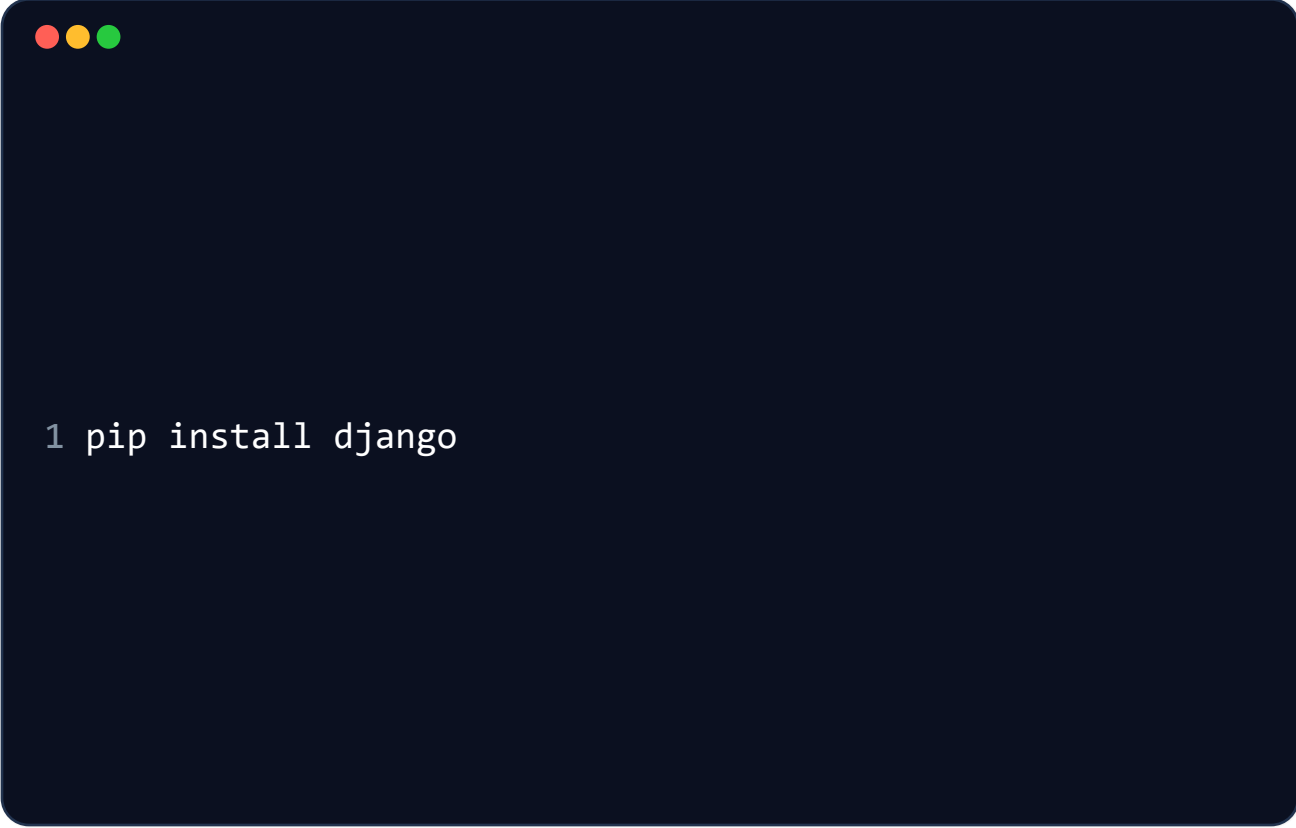
```
1 source venv/bin/activate
```

What to explain

- Use source followed by the activation script path.
- The goal is the same: enter the isolated environment.
- Again, look for (venv) in the prompt.

Install Django inside the environment

The package manager downloads the framework and its dependencies



```
1 pip install django
```

What to explain

- Run it after activating the virtual environment.
- pip downloads Django into the environment only.
- Students need internet for this step.

Verify the Django installation

Make the framework introduce itself



```
1 django-admin --version
```

What to explain

- A version number means the installation succeeded.
- If command not found appears, environment or PATH may be wrong.
- This is like a health check before project creation.

Part 3 • First Django Project

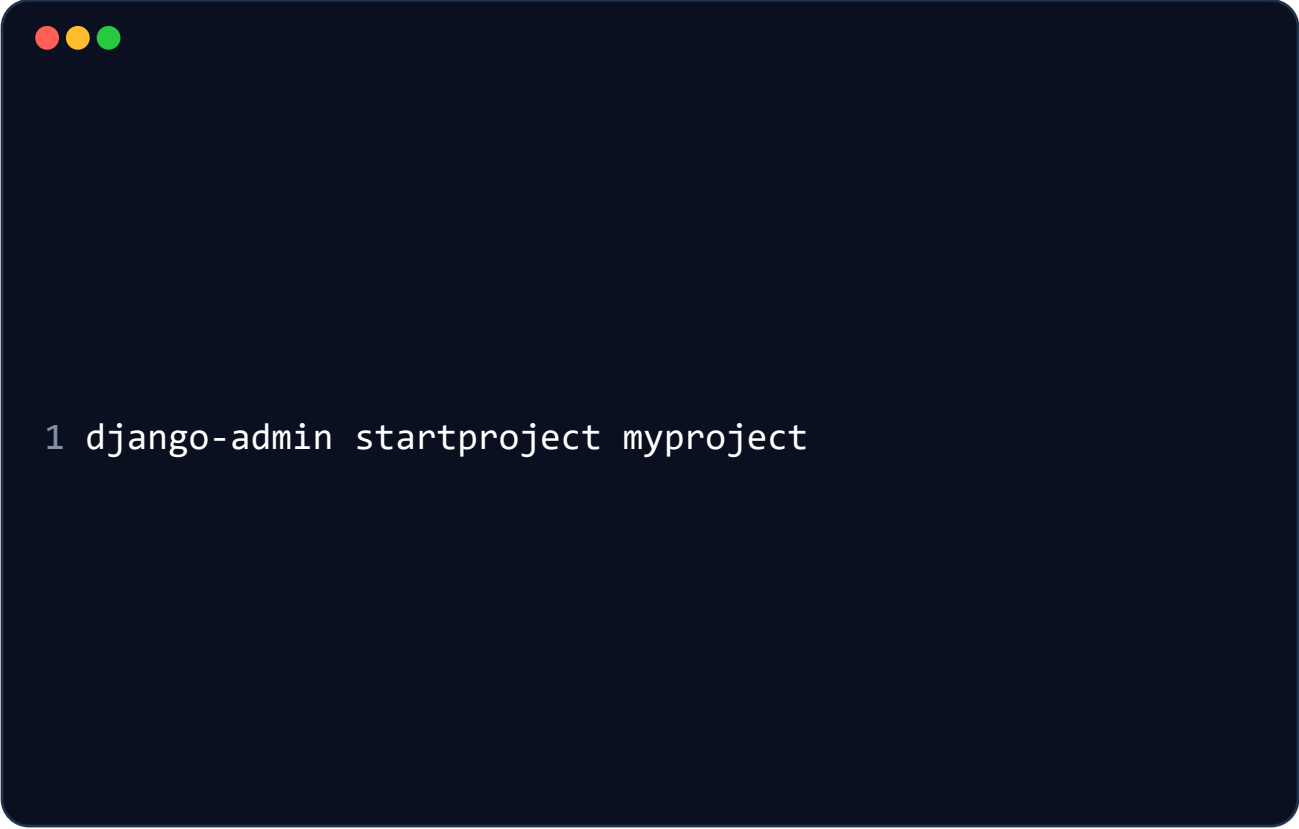
From startproject to the first successful page in the browser.

In this part, slow down and connect every command to a real classroom demonstration.

The Django logo, featuring the word "django" in a dark green, lowercase, sans-serif font. The letter 'j' is stylized with a long, curved tail that extends downwards and to the left.

Create the first project

The command that generates the initial structure

A dark-themed terminal window with three colored window control buttons (red, yellow, green) in the top-left corner. The text '1 django-admin startproject myproject' is displayed in a light-colored monospace font.

```
1 django-admin startproject myproject
```

What to explain

- django-admin is the command-line tool.
- startproject tells Django to generate a new project.
- myproject is the project name; students can change it.

What appears after startproject?

Two important levels: outer folder and inner project package

What students will see

- An outer folder named myproject
- A manage.py file
- An inner folder also named myproject
- Inside it: settings.py, urls.py, asgi.py, wsgi.py

Teacher advice

Students often get confused by the two similar names. Slow down here and point to each level clearly in the file explorer.

Initial project tree

Use this slide to point at each file with the cursor.

myproject/

Outer project
folder
Contains
manage.py and
the inner package

manage.py

Command
entry point
Used for
runserver,
migrate,
startapp

myproject/

Inner project
package
Contains settings
and URLs

Inside the inner folder

settings.py
urls.py
asgi.py
wsgi.py

What is manage.py?

The command entry point for project administration

Use cases students will repeat often

- Run the development server
- Create apps
- Make migrations
- Migrate the database
- Create admin users

Simple wording to tell the class

manage.py is not where we write the main web logic. It is the tool we use to control the project from the command line.

What is settings.py?

The central configuration file

What lives there

- Installed apps
- Database configuration
- Templates and static settings
- Language, timezone, and security-related settings

How to explain it simply

If the project were a machine, settings.py would be the control panel where many general switches are stored.

What is urls.py?

The traffic map of the project

Key role

- Maps URL paths to views or included app routes.
- Controls where each request should go.
- Grows gradually as the project grows.

Analogy

Think of it as a road map. When the browser requests `/students/`, Django checks the map to know which code should handle that request.

What are asgi.py and wsgi.py?

Deployment-related entry files

Simple explanation for beginners

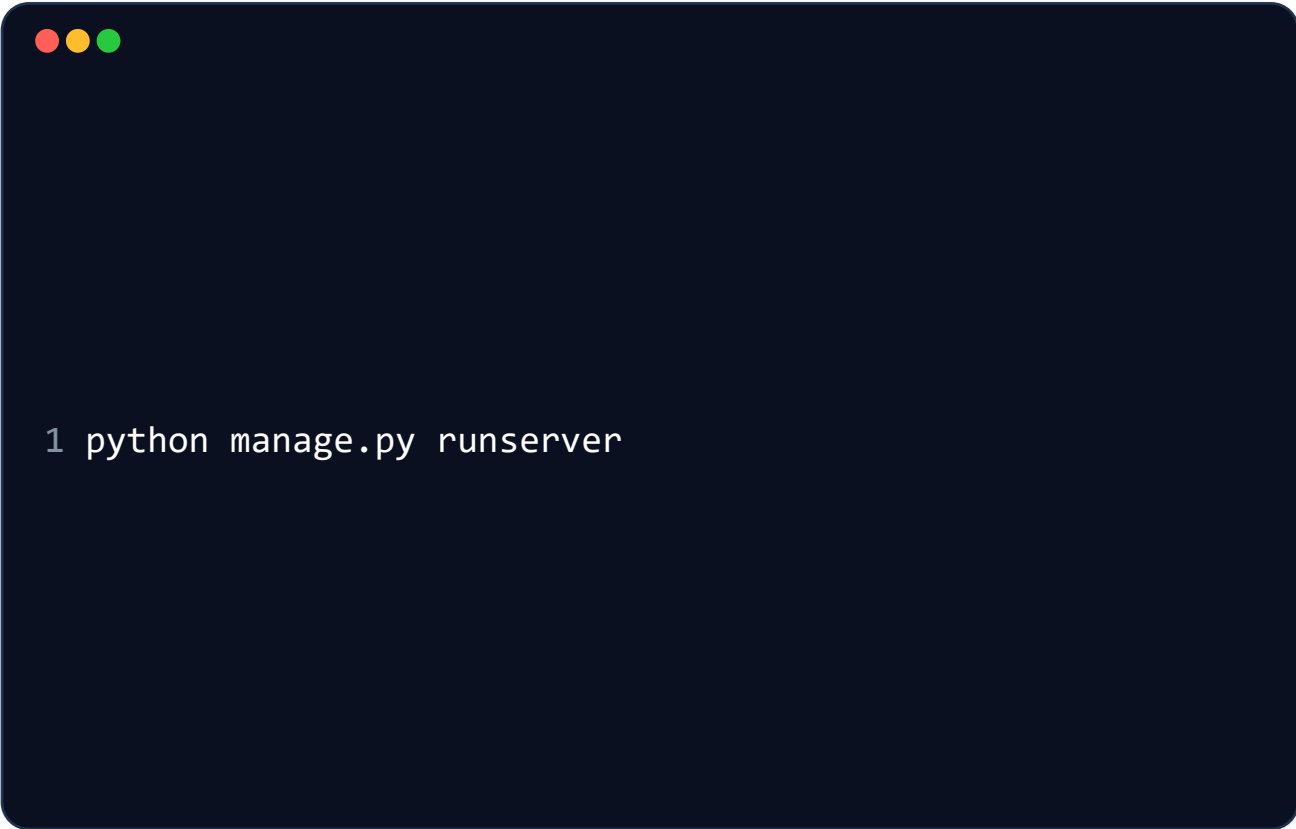
- They help servers communicate with your Django project.
- wsgi.py is the traditional interface used by many deployments.
- asgi.py is useful for newer async-capable setups.

What to tell second-year students

Do not fear these files. In the first phase of the course, students mainly need to recognize their purpose, not edit them deeply.

Run the development server

The most repeated command in daily Django work

A dark-themed terminal window with three colored window control buttons (red, yellow, green) in the top-left corner. The text '1 python manage.py runserver' is displayed in a light-colored monospace font.

```
1 python manage.py runserver
```

What to explain

- Run this command from the outer project folder.
- Django starts a local development server.
- The terminal prints an address such as 127.0.0.1:8000.

How to read the startup message

Do not skip the terminal output

Important items on the screen

- Server address
- Confirmation that no fatal error occurred
- Watching for file changes with auto-reload
- Quit command hint

Teacher note

Train students to read terminal messages. In real work, the terminal often explains exactly what is happening.

Open the browser at 127.0.0.1:8000

This is the first visible success moment for students.

What students should see

- The default Django welcome page
- A message showing the installation worked
- A clear sign that the project is alive locally

Why this moment matters

This is often the first time students feel they have launched a real web framework on their own machine.

Stop the server correctly

The standard terminal shortcut

A dark-themed terminal window with three colored window control buttons (red, yellow, green) in the top-left corner. The text "1 Ctrl + C" is displayed in a monospaced font on the first line.

```
1 Ctrl + C
```

What to explain

- This stops the running development server.
- Students should use it before changing folders or closing the terminal.
- A running server keeps the terminal busy.

Part 4 • Common Setup Problems

Turn classroom errors into teaching moments instead of panic.

In this part, slow down and connect every command to a real classroom demonstration.



Problem 1: “python is not recognized”

A classic setup issue on Windows

What it usually means

- Python is not installed, or
- Python is installed but not added to PATH.
- Sometimes the terminal needs to be reopened after installation.

Immediate classroom fix

Close and reopen terminal, verify the installation, and if needed reinstall Python with the PATH option checked.

Problem 2: packages installed in the wrong place

The student forgot to activate the environment

Symptoms

- Django installed globally instead of in the project environment
- The prompt does not show (venv)
- Commands behave differently between machines

How to teach the habit

Before any pip install command, students should look at the prompt and confirm the environment is active.

Problem 3: Port already in use

The default port 8000 may be busy

What happens

- runserver tries to use port 8000 by default.
- Another process may already be using it.
- Django will complain that the address is already in use.

Fast solution

Stop the other process if you can, or run the server on another port such as 8001.

Run the server on another port

Useful when 8000 is busy

A dark-themed terminal window with three colored window control buttons (red, yellow, green) in the top-left corner. The terminal displays a single line of text: `1 python manage.py runserver 8001`.

```
1 python manage.py runserver 8001
```

What to explain

- The last number is the desired port.
- Open the browser at the new address.
- This does not change the project itself; it only changes where the local server listens.

Part 5 • Project and App Preview

Today we preview apps so students are ready for the next lecture.

In this part, slow down and connect every command to a real classroom demonstration.

The Django logo, featuring the word "django" in a dark green, lowercase, sans-serif font. The letter 'j' is stylized with a long, curved tail that extends downwards and to the left.

Project vs App in Django

One of the most important distinctions for beginners

Project

- The whole website configuration
- Contains overall settings and global URLs
- Can include many apps

App

A smaller module focused on one feature or domain, such as students, courses, blog, or accounts. A project may contain several apps that work together.

Create the first app (preview)

We will use apps deeply in the next lecture

```
1 python manage.py startapp students
```

What to explain

- Run it from the same outer folder that contains manage.py.
- A new folder called students will be created.
- This command gives us files for views, models, admin, tests, and more.

Register the app in settings.py

Django must know which apps are installed.

What students will do later

- Open settings.py
- Find the INSTALLED_APPS list
- Add the app name such as students

Reason

If the app is not registered, Django will not fully include its models and related behavior in the project lifecycle.

INSTALLED_APPS example

A tiny code change with big importance

Explain first

- The list contains built-in Django apps already.
- We append our app name as a string.
- Pay attention to commas and quotation marks.

```
1 INSTALLED_APPS = [  
2     'django.contrib.admin',  
3     'django.contrib.auth',  
4     'django.contrib.contenttypes',  
5     'django.contrib.sessions',  
6     'django.contrib.messages',  
7     'django.contrib.staticfiles',  
8     'students',  
9 ]
```

A first Django view

Views are Python functions that receive a request and return a response.

Explain first

- The function takes request as input.
- It returns an HttpResponse object.
- This is a minimal example before templates.

```
1 from django.http import HttpResponse
2
3 def home(request):
4     return HttpResponse("Welcome to Django")
```


A first app-level URL pattern

The path decides which view should run

Explain first

- path connects a URL to a view function.
- The empty string means the root path of that app.
- This code is usually placed in students/urls.py.

```
1 from django.urls import path
2 from . import views
3
4 urlpatterns = [
5     path('', views.home),
6 ]
```

Include app URLs in the main project

Project-level URLs can delegate to app-level URLs

Explain first

- include sends a group of routes to the app.
- This keeps the project organized as it grows.
- Each app can manage its own URL file.

```
1 from django.contrib import admin
2 from django.urls import path, include
3
4 urlpatterns = [
5     path('admin/', admin.site.urls),
6     path('students/', include('students.urls')),
7 ]
```

HttpResponse is the simplest response

Useful for testing routing before templates

```
1 from django.http import HttpResponse
2
3 def about(request):
4     return HttpResponse("About page")
```

What to explain

- Good for early testing.
- Not ideal for full page design.
- Later we will use render and templates for real pages.

Templates: the next step after HttpResponse

HTML files give us structured pages instead of plain text responses.

What changes conceptually

- The view can prepare data.
- The template controls the page layout.
- This separates logic from presentation.

Why that is good

Students can keep Python logic in views and visual page structure in HTML templates. This is cleaner and easier to maintain.

Django admin: a built-in power feature

One reason many beginners love Django quickly

What the admin can do

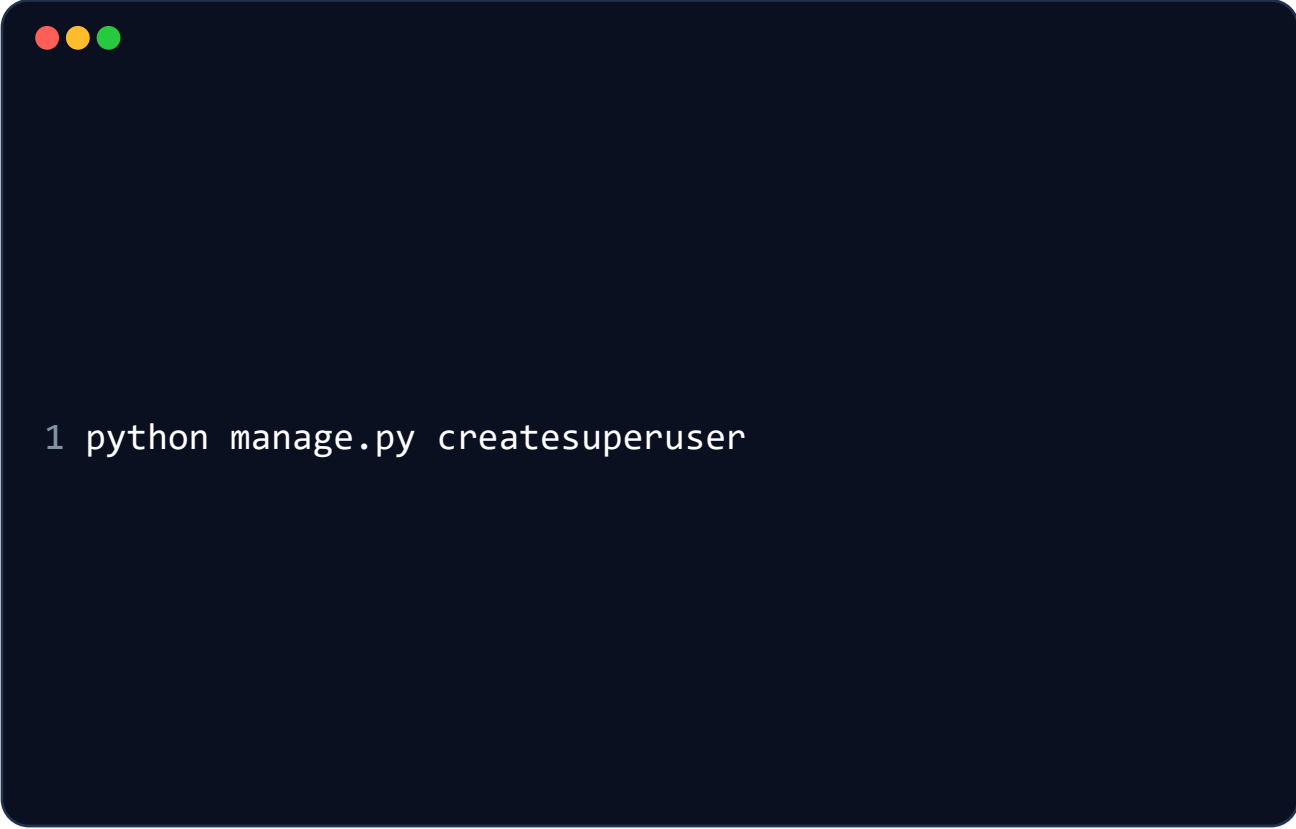
- Manage records through a ready-made interface
- Add, edit, and delete data
- Review models without building custom pages first

Teaching value

It lets students see their data in a real interface early. This creates motivation before building their own forms and dashboards.

Create an admin user

Django calls it a superuser



```
1 python manage.py createsuperuser
```

What to explain

- The terminal will ask for username, email, and password.
- This account can sign in to the admin panel.
- Students should choose memorable credentials during lab work.

What is a migration?

A controlled way to apply model changes to the database

Beginner understanding

- Models describe data structure in Python.
- Migrations translate those structural changes into database operations.
- They keep database evolution organized.

Simple analogy

Think of the model as the design on paper and the migration as the approved construction change applied to the actual building.

Generate migration files

Django inspects model changes and prepares migration instructions



```
1 python manage.py makemigrations
```

What to explain

- Creates migration files based on model changes.
- It does not yet apply them to the database.
- Think of it as preparing the plan.

Apply migrations to the database

This step updates the actual database tables



```
1 python manage.py migrate
```

What to explain

- Reads migration files and applies them.
- Creates or updates database tables.
- Needed before many features can work correctly.

End-to-end classroom workflow

From empty folder to a running local Django project

1. Check tools

Python
pip
VS Code

2. venv

Create
Activate

3. Install Django

pip install
verify version

4. startproject

Generate
project files

5. runserver

Open
browser

6. startapp preview

Prepare for
next lecture

Classroom Practice 1

Students work in pairs for 5–7 minutes

Task

- One student explains the installation sequence out loud.
- The other student types the commands in the correct order on paper or in VS Code.
- Then they swap roles.

Expected output

By the end, each pair should be able to write a correct sequence from checking Python to running the first server.

Classroom Practice 2

Identify files by role

Ask students to match

- `manage.py` → command tool
- `settings.py` → project configuration
- `urls.py` → route map
- `asgi.py` / `wsgi.py` → server entry interfaces

Teacher direction

Call random students to explain one file in one sentence only. Short answers reveal whether the concept is clear.

Mini Quiz

Use these orally or as a quick written check

Questions

- What is the purpose of a virtual environment?
- What does startproject create?
- Which file is used to run project commands?
- What command starts the development server?
- Where do we register a new app?

How to use in class

Ask students to answer in full sentences, not one-word answers. This helps them practice technical explanation, not memorization only.

Homework

Students must repeat the full setup on their own machine.

Required tasks

- Install Python if needed and verify `python --version`.
- Create and activate a virtual environment.
- Install Django.
- Create a project named `fcai_project`.
- Run the server successfully and take a screenshot.

Optional challenge

Ask students to create an app named `students` and identify the generated files, even if they do not build features yet.

Lecture recap

Five messages students should remember after class

Main takeaways

- Django is a Python web framework that gives structure.
- A working environment is the first real success condition.
- Virtual environments isolate project packages.
- startproject creates the initial project structure.
- runserver lets us view the project in the browser.

Closing line to students

If you can repeat today's setup independently, then you are ready for apps, URLs, views, and templates in the next lectures.

Preview of Lecture 6

Next time: apps, URLs, views, and the first real pages

What comes next

- Create apps with purpose
- Connect app URLs to project URLs
- Write views that return responses
- Start thinking like a Django developer

Motivation note

Today we prepared the lab. Next lecture we begin the real application flow inside that lab.